Blocks Project

Description

Write a graphical program BlocksMain.java that draws moving blocks on the screen. The graphical user interface allows the user to interact through key presses and mouse clicks as outlined in this documentation. When the program runs, 20 blocks should be displayed on the screen (stored in a blocks list). You may use starter code from a previous project or begin from scratch.

The graphical user interface displays the blocks and allows the user to manipulate them.

- If the user clicks the **left mouse button** while the mouse cursor points at a tile, that tile is moved to the very top of the z-ordering (the end of the block list). MouseEvent.BUTTON1, MouseEvent.BUTTON2 and MouseEvent.BUTTON3 are for left, middle and right click respectively.
- If the user clicks the **left mouse button** and holds down the **Shift key** while the mouse cursor points at a block, that block is moved to the very bottom of the z-ordering (the start of the block list). Given a MouseEvent object m, m.isShiftDown() will be useful.
- If the user clicks the **right mouse button** while the mouse cursor is pointing at a block, that block is removed from the block list and disappears from the screen.
- If the user clicks the **right mouse button** and holds the **Shift** key, all blocks that occupy that pixel are removed from the block list and disappear from the screen.
- If the user types the **N key** on the keyboard, a new randomly positioned block is created and added to the screen.
- If the user types the **S key** on the keyboard, the block order and location are randomly rearranged (shuffled) for all blocks.
- If the user clicks the **middle mouse button**, all blocks that occupy that pixel are exploded (explained later in this document).
- If the user types the **E key** on the keyboard, all blocks are exploded.
- If the user types the **I (letter i) key** on the keyboard, all blocks are imploded.
- If the user clicks the **'start' button**, the blocks will move around the screen.
- If the user clicks the **'stop' button**, the blocks will stop moving.
- If the user clicks **'step' button**, the blocks move once.
- If the user clicks the **'reset' button**, all blocks are removed.

Implementation Details

You'll be reusing the Block class you implemented earlier this year. Add two integer attributes **dx** (delta x for x speed) and **dy** (delta y for y speed) randomly set –2, -1, 1 or 2 with equal probability. Add appropriate constructors, accessors and mutators to the Block class. Add a **move** method that updates x and y by dx and dy respectively.

Your **BlockManager** class should store a list of blocks. The various methods listed below will cause changes to the contents of that list. The following sections describe in detail each method you must implement in your **BlockManager** class. For any methods that accept objects as parameters, you may assume that the value passed is not null or otherwise invalid.

public **BlockManager**()
Instantiate the list. Initially it stores no blocks.

public void **addBlock**(Block block)
Add the block to the list of blocks.

public void **drawAll**(Graphics g)
Draw all of blocks.

public void **raise**(int x, int y)
Invoked when the user left clicks and passes the x/y coordinates of the mouse click. If these coordinates touch a block, then move it to the very top (end) of the list.

public void **lower**(int x, int y)
Invoked when the user shift-left clicks. Moves only the topmost block, if applicable, to the very bottom (beginning) of the list.

public void **delete**(int x, int y)
Invoked when the user right clicks. If these coordinates touch a block, then delete the topmost block from the list.

public void **deleteAll**(int x, int y)
Invoked when the user shift-right clicks. If these coordinates touch any block, then delete the block from the list.

public void **moveAll**(int width, int height)
Moves all blocks by updating the x and y by the xSpeed and ySpeed respectively. If any block collides with the bounding region, then set the respective speed to speed *= -1. The blocks should bounce off the walls. One caveat is that exploding blocks move based on an angle and do not bounce off walls. Remove any exploding blocks that are out of range.

public void **shuffle**(int width, int height)
Shuffles the list and relocates all blocks. Each relocated block must entirely fit into the given width and height parameters.

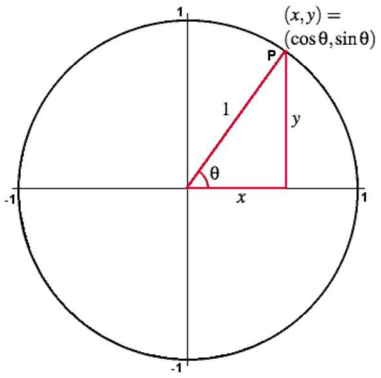
public void **explode**(int x, int y)
Invoked when the user middle clicks. If these coordinates touch any block, then delete the block from the list and create exploding blocks. An exploding block will be 1 pixel and will radiate outwards from the center. Create len * width exploding blocks from the clicked block. An exploding block is a child of block with 3 double attributes, x, y and angle.

public void **explodeAll**()
Invoked when the user presses the 'E' key. All non-exploding blocks will be removed and turned into exploding blocks. b instanceof ExplodingBlock will be useful.

public void **implodeAll**()
Invoked when the user presses the 'I' key. All non-exploding blocks will be removed and turned into exploding blocks. These exploding blocks will radiate inward toward the center of the original block.

public void **clear**()
Removes all blocks from the blocks list.

ExplodingBlock

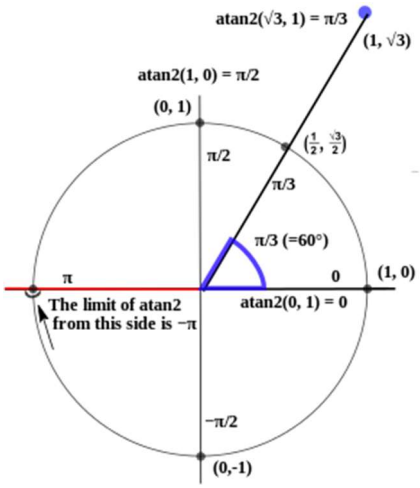


Extend the Block class and create an ExplodingBlock class. Add three double attributes x, y angle or theta (in radians). Add an appropriate constructor (be sure to call super), an accessor and mutator for the angle. Override draw() and the setters/getters for x and y. The code is identical, but the doubles will need to be casted to an int. Override the move method. An exploding block moves based on the angle. Use the cosine and sine functions to increment x and y respectively.

Development Strategy and Hints

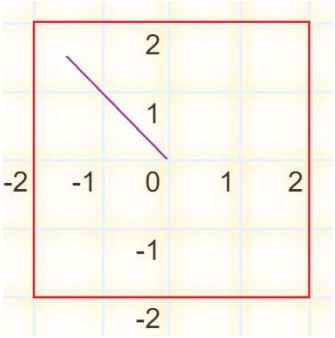
Developing your code in stages and knowing how to test your solutions will be beneficial to your success. One of the most important techniques for professional developers is to write code in stages ("iterative enhancement" or "stepwise refinement") rather than trying to do it all at once. This includes testing for correctness at each stage before moving to the next one. Start by writing empty "stub" versions of all the required methods so that the BlockManager class will compile. Code the easiest ones first and run the program to make sure that you can see the blocks appear on the screen. Add click-related methods one at a time and test each one individually to be sure it works before moving on to the next. Save explode and implode for last.

Explode/Implode



The angles for explode can be calculated using basic trigonometry learned in a typical geometry class. Java's built-in utility Math class provides an easier alternative. atan(y, x) returns the angle *theta* from the conversion of rectangular coordinates (x, y) to polar coordinates (r, *theta*). This method computes the phase *theta* by computing an arc tangent of y/x in the range of *-pi* to *pi*. If x > 0, then atan2(y, x) == arctan(y/x) == θ. If x < 0, then arctan(y/x) is diametrically opposite and the angles must be adjusted. atan2(y/x) takes care of the adjustment for you. In Java, sin, cos and atan all use radians so don't convert to degrees.

In explode, create exploding blocks that are 1x1 (a single pixel). The instantiated exploding blocks will move away from the center. The number of objects created should be width * height. You need to calculate the angle formed by the direction the block will move with the x-axis. Given a 4 by 4 block, the center would be at (2, 2). The top left exploding block would be at (0, 0) with a center of (.5, .5). Then the angle is Math.atan2(py - cy, px - cx) where py/px represents the center of the new exploding block and cy/cx represents the center of the block to be blown up. We subtract to account for the offset.



Angles formed by a 4x4 block into 16 exploding blocks.

-135.0	-108.4	71.6	-45.0
-161.6	-135.0	-45.0	-18.4
161.6	135.0	45.0	18.4
135.0	108.4	71.6	45.0

The chart shows the angles calculated. Notice how opposite angles (from center) differ by 180 degrees and all angles account for the fact that y values increase downwards in Java's coordinate system. In implode, the blocks should move in the opposite direction.

Style Guidelines and Grading

You should follow good general Java style guidelines such as: appropriately using control structures like loops and if/else statements; avoiding redundancy using techniques such as methods, loops, and factoring common code out of if/else statements; properly using indentation, good variable names, and types. And not having any lines of code longer than 100 characters in length and not having any run-on methods. A good rule of thumb for methods is that they should fit on the screen. You should **comment** your code with a heading at the top of your class with your name, period, and a description of the overall program. Also place a comment heading on top of each method, and a comment on any complex sections of your code. Comment headings should be written in your own words, explaining each method's behavior, parameters, return values, and assumptions made by your code, as appropriate.

Sample Execution

